AI / Data Science
Interview Questions & Answers

Interview-ready answers • Agentic RAG • Document scoring • Scalability • Optimization

1. Agent Orchestration in a Healthcare Agentic RAG System

Question: How are you orchestrating the agents in your healthcare Agentic RAG system?

I use a multi-agent architecture where each agent has a specific responsibility. A router agent first understands the user's query and decides which agent or tool should handle it. The relevant agents then perform retrieval, analysis, and validation. Finally, a response agent combines the results and generates the final answer. I use LangGraph for orchestration because it lets me control workflow steps and state between agents.

Keywords: Multi-agent, Router, RAG, LangGraph, Workflow

2. Ensuring Correct Communication and Responses

Question: How do you ensure that all agents communicate properly and provide correct responses?

I use structured inputs and outputs between agents. I define a fixed schema with Pydantic so every agent returns the expected information. I validate retrieved information, check generated responses against source documents, and use confidence thresholds where appropriate. In healthcare, I preserve citations and evidence so the final answer is grounded in reliable sources.

Keywords: Pydantic, Validation, Grounding, Citations, Confidence

3. Technologies Apart from RAG

Question: Apart from RAG, what other tools or technologies are you using?

Apart from RAG, I use LLMs, LangGraph or LangChain for orchestration, Hugging Face for models, vector databases for embeddings, Python for data processing, FastAPI for APIs, and Docker for deployment. I also use monitoring and evaluation tools to track latency, accuracy, errors, and LLM costs.

Keywords: LLM, LangGraph, Hugging Face, Vector DB, FastAPI, Docker

4. Evaluating the Healthcare Agentic RAG System

Question: How did you evaluate the healthcare Agentic RAG system?

I evaluate the system at retrieval and generation levels. For retrieval, I check Precision, Recall, and Mean Reciprocal Rank (MRR). For generated answers, I evaluate faithfulness, relevance, and whether claims are supported by retrieved documents. I also measure latency, cost, error rate, and response accuracy using a predefined test dataset.

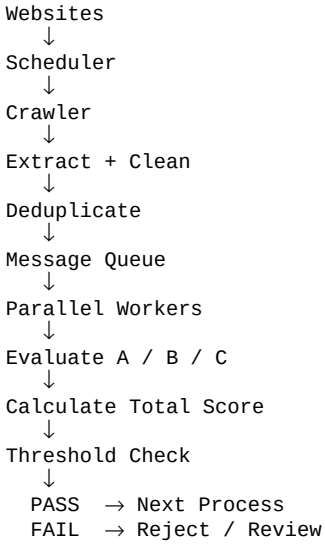
Keywords: Precision, Recall, MRR, Faithfulness, Relevance, Latency, Cost

5. Website Document Scoring System

Question: Websites provide documents periodically. Each document must be evaluated against criteria A, B, and C, scored 0–2. Calculate the total and process only documents meeting a threshold. How would you design it?

I would design an automated document-processing pipeline. A scheduler periodically fetches information from the websites. Documents are extracted, cleaned, and deduplicated, then sent to a processing queue. Multiple workers evaluate criteria A, B, and C, assigning each a score from 0 to 2. The total score is compared with a predefined threshold. Documents that meet the threshold continue to the next process; others are rejected or stored for review.

Architecture:

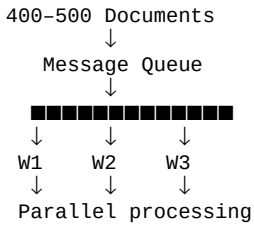


Tools: Python, Scrapy / Playwright, Airflow, Pub/Sub / Kafka, LangGraph, PostgreSQL, Cloud Storage

6. Handling 400–500 Documents Arriving at Once

Question: What if 400–500 documents arrive at the same time?

I would avoid sequential processing. I would place documents in a message queue and process them with multiple asynchronous workers. The queue distributes work, while bounded concurrency controls API rate limits, memory use, and cost. I would also add retries, timeouts, idempotency, and dead-letter handling for failed jobs.

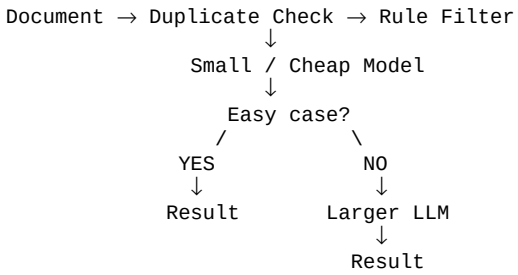


Keywords: Async, Parallel Processing, Queue, Workers, Bounded Concurrency

7. Making the System Faster and Cheaper

Question: How can you make the system faster and cheaper?

I would remove duplicate documents, cache repeated results, and apply rule-based filtering before calling an LLM. I would route simple cases to a smaller, cheaper model and reserve a larger model for difficult cases. I would use asynchronous processing and batch inference where supported, reduce prompt size by sending only relevant text, and monitor cost and latency. For self-hosted models, quantization and optimized inference frameworks such as vLLM may help.



Keywords: Caching, Small LLM, Model Routing, Batch Inference, Quantization, vLLM

Important Keywords to Remember

Topic	Keywords
Agent orchestration	Multi-agent, LangGraph, Router
Agent communication	Pydantic, Structured Output, Validation
Technologies	LLM, RAG, Vector DB, Hugging Face, FastAPI
Evaluation	Precision, Recall, MRR, Faithfulness
Document system	Crawler, Queue, Workers, Threshold
400–500 documents	Async, Parallel Processing, Bounded Concurrency
Optimization	Caching, Small LLM, Model Routing, Batch, Quantization

Strong Interview Closing Line

> “My main focus is to make the system accurate, scalable, cost-efficient, and observable.”

Interview tip: Be precise about what you personally implemented. If a tool or metric was only part of your design proposal, describe it as a planned approach rather than claiming it is already in production.